

Código completo – parte 1

```
main.py > @atualizaFuncionario
1 > from flask import Flask, request, jsonify
2 > import pymysql
3 > from flask_mail import Mail, Message
4 > import random
5
6 app = Flask(__name__)
7
8 > def conectaDB():
9 >     db = pymysql.connect(
10 >         host='localhost',
11 >         database='banco_funcionarios',
12 >         user='root',
13 >         passwd='',
14 >     )
15 >     return db
16
17
18
19 ## CREATE -- CADASTRO DO FUNCIONARIO
20 @app.route('/cadastraFuncionario', methods=['POST'])
21 > def cadastraFuncionario():
22 >     dadosRecebidos = request.get_json()
23
24     nome = dadosRecebidos["nomeFuncionario"]
25     telefone = dadosRecebidos["telefoneFuncionario"]
26     idUsuarioFK = dadosRecebidos["idUsuarioFK"]
27     banco = conectaDB()
28     cursor = banco.cursor()
29
30     sql = f"INSERT INTO funcionario(nome, telefone, id_usuario_fk) VALUES ({nome}, {telefone}, {idUsuarioFK})"
31     try:
32         cursor.execute(sql)
33         banco.commit()
34         response = {"mensagem": "Cadastrado com sucesso", "codigo": 200}
35     except Exception as e:
36         response = {"mensagem": "Erro ao cadastrar funcionario", "codigo": 400, "erro": str(e)}
37     finally:
38         banco.close()
39
40     return jsonify(response)
41
```

Quarto passo

Implemente o código



Código completo – parte 2

Profº: Joseph Donald

```
42 # rota para atualização dos dados do funcionário nomeada como /atualizaFuncionario, utilizando o método PUT
43 @app.route('/atualizaFuncionario', methods=['PUT'])
44 def atualizaFuncionario():
45     dadosRecebidos = request.get_json()
46     idFuncionario = dadosRecebidos["idFuncionario"]
47     nomeFuncionario = dadosRecebidos["nomeFuncionario"]
48     telefone = dadosRecebidos["telefoneFuncionario"]
49     idUsuarioFK = dadosRecebidos["idUsuarioFK"]
50
51     banco = connectDB()
52     cursor = banco.cursor()
53
54
55     sql = f"UPDATE funcionario SET nome = '{nomeFuncionario}', telefone = '{telefone}' WHERE id_funcionario = {idFuncionario} AND id_usuario_fk = {idUsuarioFK}"
56     try:
57         cursor.execute(sql)
58         banco.commit()
59         response = {"mensagem": "Atualizado com sucesso", "codigo": 200}
60     except Exception as e:
61         response = {"mensagem": "Erro ao atualizar funcionário", "codigo": 400, "erro": str(e)}
62     finally:
63         banco.close()
64
65     return jsonify(response)
66
67
68 ## READ - LER OS FUNCIONARIOS DO BANCO somente do usuário logado, utilizando o método GET e recebendo o id do usuário como parâmetro na URL
69 @app.route('/lerFuncionario/<int:id_usuario>', methods=['GET'])
70 def lerFuncionario(id_usuario):
71     banco = connectDB()
72     cursor = banco.cursor()
73
74     sql = f"SELECT * FROM funcionario WHERE id_usuario_fk = {id_usuario};"
75     try:
76         cursor.execute(sql)
77         resultado = cursor.fetchall()
78         response = {"mensagem": "Funcionários lidos com sucesso", "codigo": 200}
79     except Exception as e:
80         response = {"mensagem": "Erro ao ler funcionarios", "codigo": 400, "erro": str(e)}
81     finally:
82         banco.close()
83
84     listaFuncionario = []
85
86     for funcionario in resultado:
87         listaFuncionario.append(
88             {
89                 "id": funcionario[0],
90                 "nomeFuncionario": funcionario[1],
91                 "telefoneFuncionario": funcionario[2],
92                 "idusuarioFK": funcionario[3]
93             }
94         )
95
96     return jsonify(listaFuncionario, response)
97
```

Código completo – parte 3

Profº: Joseph Donald

```
98 # Rota para deletar um funcionário utilizando o método DELETE
99 @app.route('/deletaFuncionario', methods=['DELETE'])
100 def deletaFuncionario():
101     dadosRecebidos = request.get_json()
102     idFuncionario = dadosRecebidos["idFuncionario"]
103     idUsuarioFk = dadosRecebidos["idUsuarioFk"]
104
105     banco = conectaDB()
106     cursor = banco.cursor()
107
108     sql = f"DELETE FROM funcionario WHERE id_funcionario = {idFuncionario} AND id_usuario_fk = {idUsuarioFk}"
109     try:
110         cursor.execute(sql)
111         banco.commit()
112         response = {"mensagem": "Deletado com sucesso", "codigo": 200}
113     except Exception as e:
114         response = {"mensagem": "Erro ao deletar funcionário", "codigo": 400, "erro": str(e)}
115     finally:
116         banco.close()
117
118     return jsonify(response)
119
120
121 ## CREATE - ROTA PARA CADASTRAR UM USUÁRIO
122 @app.route('/cadastraUsuario', methods=['POST'])
123 def cadastraUsuario():
124     dadosRecebidos = request.get_json()
125     nome = dadosRecebidos["nomeUsuario"]
126     email = dadosRecebidos["emailUsuario"]
127     senha = dadosRecebidos["senhaUsuario"]
128
129     banco = conectaDB()
130     cursor = banco.cursor()
131
132     sql = f"INSERT INTO usuario(nome, email, senha) VALUES ({nome}, '{email}', '{senha}')"
133     try:
134         cursor.execute(sql)
135         banco.commit()
136         response = {"mensagem": "Cadastrado com sucesso", "codigo": 200}
137     except pymysql.err.IntegrityError as e:
138         response = {"mensagem": "Email já cadastrado", "codigo": 400, "erro": str(e)}
139     finally:
140         banco.close()
141
142     return jsonify(response)
143
```



Código completo – parte 4

Profº: Joseph Donald

```
145 ## LOGIN - ROTA PARA REALIZAR O LOGIN DO USUÁRIO recolhendo todas as informações do usuário para realizar a autenticação
146 @app.route('/loginUsuario', methods=['POST'])
147 def loginUsuario():
148     dadosRecebidos = request.get_json()
149     email = dadosRecebidos["emailUsuario"]
150     senha = dadosRecebidos["senhaUsuario"]
151
152     banco = conectaDB()
153     cursor = banco.cursor()
154
155     sql = f"SELECT id_usuario, nome, email, senha FROM usuario WHERE email = '{email}' AND senha = '{senha}'"
156     try:
157         cursor.execute(sql)
158         resultado = cursor.fetchone()
159         response = {"mensagem": "Login realizado com sucesso", "codigo": 200, "usuario": {
160             "id" : resultado[0],
161             "nome" : resultado[1],
162             "email" : resultado[2]
163         }}
164     except Exception as e:
165         response = {"mensagem": "E-mail ou senha incorretos", "codigo": 401, "erro": str(e)}
166     finally:
167         banco.close()
168
169     return jsonify(response)
170
```



Código completo – parte 5

Profº: Joseph Donald

```
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215

## rota de recuperação de senha através do Flask-Mail, utilizando o método POST para enviar um e-mail de recuperação com um código temporário de confirmação
@app.route('/recuperaSenha', methods=['POST'])
def recuperaSenha():

    global codigo_recuperacao_gerado

    dadosRecebidos = request.get_json()
    email = dadosRecebidos["emailUsuario"]

    banco = conectaDB()
    cursor = banco.cursor()

    sql = f"SELECT id_usuario, nome, email FROM usuario WHERE email = '{email}'"
    try:
        cursor.execute(sql)
        resultado = cursor.fetchone()
        if resultado:
            app.config['MAIL_SERVER'] = 'smtp.gmail.com'
            app.config['MAIL_PORT'] = 587
            app.config['MAIL_USE_TLS'] = True
            app.config['MAIL_USERNAME'] = 'josephdonald@gmail.com'
            app.config['MAIL_PASSWORD'] = 'coyq fwjt wf01 kxtv'
            mail = Mail(app)

            msg = Message("Recuperação de Senha", sender="josephdonald@gmail.com", recipients=[email])
            codigo_recuperacao_gerado = random.randint(100000, 999999)
            msg.body = f"Olá seu código de recuperação de senha é: {codigo_recuperacao_gerado}"

            mail.send(msg)
            response = {"mensagem": "E-mail de recuperação enviado com sucesso", "codigo": 200, "usuario": {
                "id": resultado[0],
                "nome": resultado[1],
                "email": resultado[2]
            }}
        else:
            response = {"mensagem": "E-mail não encontrado", "codigo": 404}
    except Exception as e:
        response = {"mensagem": "Erro ao recuperar senha", "codigo": 400, "erro": str(e)}
    finally:
        banco.close()

    return jsonify(response)
```




Código completo – parte 6

Profº: Joseph Donald

```
215 # Rota para atualizar a senha do usuário utilizando o método PUT, recebendo o email do usuário e a nova senha
216 @app.route('/atualizaSenha', methods=['PUT'])
217 def atualizaSenha():
218     dadosRecebidos = request.get_json()
219     email = dadosRecebidos["emailUsuario"]
220     nova_senha = dadosRecebidos["novaSenha"]
221     codigo_recuperacao_recebido = dadosRecebidos["codigoRecuperacao"]
222
223     print("Código de recuperação recebido:", codigo_recuperacao_recebido)
224     print("Código de recuperação gerado:", codigo_recuperacao_gerado)
225
226     if codigo_recuperacao_recebido != codigo_recuperacao_gerado:
227         return jsonify({"mensagem": "Código de recuperação inválido", "codigo": 400})
228
229     banco = conectaDB()
230     cursor = banco.cursor()
231
232     sql = f"UPDATE usuario SET senha = '{nova_senha}' WHERE email = '{email}';"
233     try:
234         cursor.execute(sql)
235         banco.commit()
236         response = {"mensagem": "Senha atualizada com sucesso", "codigo": 200}
237     except Exception as e:
238         response = {"mensagem": "Erro ao atualizar senha", "codigo": 400, "erro": str(e)}
239     finally:
240         banco.close()
241
242     return jsonify(response)
243
244 if __name__ == "__main__":
245     app.run(debug=True)
246
247
248
```